# Experiments On A Robotic Car: Light Seeking, Line Following, Obstacle Avoidance, And Environmental Sensing



*by*

Yuwei Zhao, Haohao Qi, Yilin Zhang

Submitted for the

*Final report*

ROBOTICS

SCHOOL OF MATHEMATICAL AND COMPUTER SCIENCES

HERIOT-WATT UNIVERSITY

June 2025

# Abstract

In accordance with the Academic Regulations, the thesis must contain an abstract preferably not exceeding 200 words, bound in to precede the thesis. The abstract should appear on its own, on a single page. The format should be the same as that of the main text. The abstract should provide a synopsis of the thesis and shall state clearly the nature and scope of the research undertaken and of the contribution made to the knowledge of the subject treated. There should be a brief statement of the method of investigation where appropriate, an outline of the major divisions or principal arguments of the work and a summary of any conclusions reached. The abstract must follow the Title Page.

# Contents


**1 Introduction** **1**
- 1.1 Our Car Lobster . . . 1
- 1.2 Four Aims of the Laboratory . . . 1
- 1.3 Result Summary . . . 2

**2 Experiments** **3**
- 2.1 Light Seeking . . . 3
  - 2.1.1 Theory and Technical Background . . . 3
  - 2.1.2 Method . . . 4
  - 2.1.3 Results . . . 5
- 2.2 Line Following . . . 5
  - 2.2.1 Theory and Technical Background . . . 5
  - 2.2.2 Method . . . 5
  - 2.2.3 Results . . . 7
- 2.3 Obstacle Avoidance . . . 8
  - 2.3.1 Theory and Technical Background . . . 8
  - 2.3.2 Method . . . 9
  - 2.3.3 Results . . . 11
- 2.4 Environment Sensing . . . 11
  - 2.4.1 Theory and Technical Background . . . 11
  - 2.4.2 Method . . . 11
  - 2.4.3 Results . . . 12

**3 Conclusions** **13**

**A Contribution Statement** **14**

**B Source Code** **15**
- B.1 Lab 3: Light Seeking . . . 15
  - B.1.1 lightToward.cpp . . . 15
- B.2 Lab 4: Line Following . . . 17
  - B.2.1 dataTrans.h . . . 17
  - B.2.2 Motion_setting.h . . . 17
  - B.2.3 Motion_control.h . . . 18
  - B.2.4 main.cpp . . . 20
- B.3 Lab 7: Obstacles Avoiding . . . 21
  - B.3.1 Motion_setting.h . . . 21
  - B.3.2 Motion_control.h . . . 22
  - B.3.3 Servo_control.h . . . 23
  - B.3.4 Ultrasound_control.h . . . 24
  - B.3.5 main.cpp . . . 24
- B.4 Lab 8: Environment Measurement . . . 26
  - B.4.1 MotionDetect.h . . . 26
  - B.4.2 MotionSetting.h . . . 26
  - B.4.3 MotionControl.h . . . 27

B.4.4 FactorDetect.h . . . . . 28
B.4.5 LCDDemonstrate.h . . . . . 29
B.4.6 main.cpp . . . . . 30

**References** **32**

## Glossary

**IR** Infrared Transducer.

**LDR** Light-Dependent Resistor.

**Lobster** Bionic Lobster.

**PID** Proportional-Integral-Derivative.

**PWM** Pulse-Width Modulation.

# Chapter 1

# Introduction

## 1.1 Our Car Lobster

The main aim of this laboratory is to build our own robotic car named "**Bionic Lobster (Lobster)**", abbreviated as "**Lobster**". Its name comes from the shape, which in our opinion is a good sigh of strength and speed. Haohao Qi is responsible for our car's shape and color design. She measures its internal dimensions and builds a 3D model using Blender. Yuwei Zhao operates a 3D printer with permission from his club.

In southern China, lobsters are considered a delicious delicacy. When harvesting them in the fields, locals find them difficult to catch due to their speed. Similarly, we hope our robotic car, Lobster, will be fast and robust.

## 1.2 Four Aims of the Laboratory

We have done four contributions to it, which are recorded in Lab 3 (light seeking), Lab 5 (line following), Lab 7 (obstacle avoidance) and Lab 8 (environment sensing). Thus, Lobster could have the following functionalities:

1. Follow the light on the floor and adjust its direction toward the light source.
2. Follow a 1 to 3 cm wide black line on the floor. The line can be straight, curved, or quarter-shaped.
3. Move straight and avoid obstacles. When reaches obstacles such as walls, Lobster will turn to another direction to avoid collisions.
4. Measure temperature, humidity and atmospheric pressure, display the readings and adjust motor speed accordingly.

If Lobster has installed light seeking module, then it cannot track lines any more, and the same applies to the other two functionalities. In short, only one of these four modules works at a time.

## 1.3 Result Summary

Basic approaches of the four experiments have been successfully completed.

We used two Light-Dependent Resistor (LDR)s to detect changes in light intensity. Lobster can recognize a straight white light in the front of it within **15 cm** long and **10 cm** wide.

Proportional-Integral-Derivative (PID) control was creatively applied in line tracking. We used TCRT5000 Infrared Transducer (IR) sensor feedback to distinguish between black and non-black surfaces on the ground. Interestingly, we found that sensor feedback exhibits a linear response while in the middle of black and white, which enabled the implementation of PID control. As a result, Lobster is able to move at a speed of **0.5 m/s** in straight lines, and **0.3 m/s** in curved ones.

For obstacle avoidance, Lobster was placed in a $3 \times 3\,\mathrm{m}^2$ square place with walls around. HC-SR04 ultrasonic distance sensor was used in this experiment to measure mainly front distance. Additionally, SG90 servo motor was applied to change HC-SR04's direction to avoid a side crash. Lobster passes a **1-minute** test in is square place without any collisions.

In the eighth laboratory experiment, we used a BME280 sensor to measure humidity, atmospheric pressure, and temperature. The collected data is displayed on a small screen mounted on Lobster's back. Based on these measurements, Lobster can determine whether it is on the ground or in the air. Accordingly, its wheel rotation speed is adjusted based on the detected state.

# Chapter 2

# Experiments

## 2.1 Light Seeking

### *2.1.1 Theory and Technical Background*

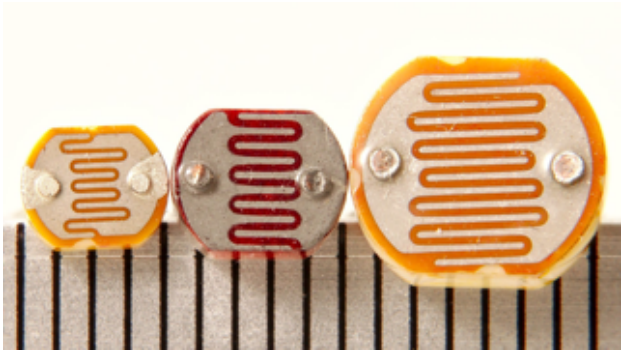


Figure 2.1: Photoresistors.Resource: makeabilitylab.github.io/physcomp

The operating principle of LDRs is rooted in the photoelectric properties of semiconductor compounds. When exposed to light, materials like cadmium sulfide (CdS) undergo photon absorption that generates electron-hole pairs within their crystalline structure. As illumination intensifies, this photogeneration effect produces a greater number of charge carriers, thereby enhancing the material's conductivity. Consequently, LDRs exhibit an inverse relationship between incident light intensity and electrical resistance-a characteristic that makes them valuable for light-sensing applications. This nonlinear response proves particularly sensitive in the 10–100 lux range, where a 20% increase in light intensity causes approximately 60% resistance drop.

LDRs in our system demonstrate a measurable resistance range from 1 kΩ under direct sunlight to 1.5 MΩ in complete darkness, as recorded by our multimeter.

We use two DC gearbox motors (TT motors) to control Lobster's movement, which typically requires Pulse-Width Modulation (PWM) for speed regulation.

The three key parameters of PWM control are frequency, duty cycle, and resolution, defined as follows:

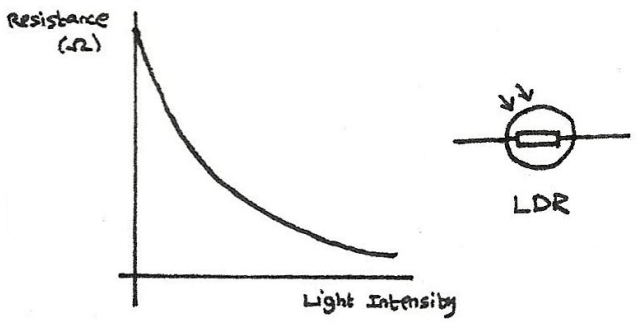



Figure 2.2: LDRs' Intensity-Resistance curves.Resource: revise.im

$$\begin{cases} \text{Freq} = \frac{1}{T_S} \\ \text{Duty} = \frac{T_{\text{ON}}}{T_S} \\ \text{Reso} = \frac{1}{T_S} \end{cases} \tag{2.1}$$

where *Freq*, *Duty*, and *Reso* represent the frequency, duty cycle, and resolution of the PWM signal, $T_S$ is the total period of one PWM cycle, and $T_ON$ is the high-level (active) time within the cycle.

The duty cycle is the most critical parameter. For example, a 50% duty cycle corresponds to approximately half of the motor's full speed, while a 100% duty cycle drives the motor at maximum speed.

These characteristics directly influence Lobster's light-seeking behavior.

### *2.1.2 Method*

With *Arduino*'s help, glspwm control can be simply done by setting motor's voltage. In package, it is 0 to 255.

Two LDRs were mounted on the left and right front sides of Lobster, each returning a sensor reading between 0 and 1023. Then, the LDR values were scaled down by a factor of 4 to match the motor input range. Lobsters steering direction was determined by the difference in illumination between the two LDRs: if the left-side reading was lower than the right, the right motors voltage was reduced (and vice versa), causing Lobster to turn toward the brighter side. Additionally, if both sensors detected insufficient light, set to over 1019, the motors would stop entirely, preventing movement in darkness. It can be described in pseudocode as follows:

```
1 if(Left < 1019 || Right < 1019) {
2   if(Gap >= expect_Gap) {
```

```
    if(Left < Right) {
      turnLeft();
    } else {
      turnRight();
    }
  } else {
    goStraight();
  }
} else {
  stop();
}
```

### *2.1.3 Results*

In the end, we managed to set straight speed at 120. Lobster passed the test twice. On the first attempt, its turning speed was set too fast, which caused uncontrollable swaying, and our teacher was not satisfied with that performance. On the second attempt, it performed very well, scoring 29 out of 30, losing 1 point for requiring two attempts.

## 2.2 Line Following

### *2.2.1 Theory and Technical Background*

Shown in figure 2.3, *TCRT5000* serves as the cornerstone in this experiment. This reflective optoelectronic device integrates both an infrared emitter (950nm photodiode) and receiver (NPN phototransistor) in a compact 4-pin package, distinguished by its blue emitter and black receiver components. [1]

During operation, the photodiode transmits modulated infrared signals while the phototransistor detects reflected radiation. The sensor's daylight-blocking optical filter ensures reliable operation under ambient lighting conditions.

### *2.2.2 Method*

The IR sensor voltage coming back was measured as $4.15 \pm 0.1\,\text{V}$ on black lines, and $0.15 \pm 0.1\,\text{V}$ on white ground. We observed a linear transition region that in the middle of black and white surfaces, enabling analog position detection suitable for PID control.

The linear voltage gradient permits error quantification for PID control. PID combines proportional (P), integral (I) and derivative (D) components. To implement PID control, there should be a reference value (setpoint) as the control target, standing for adjustment goal. Continuous measurement of the process variable to get the most recent value and then calculate the *error*

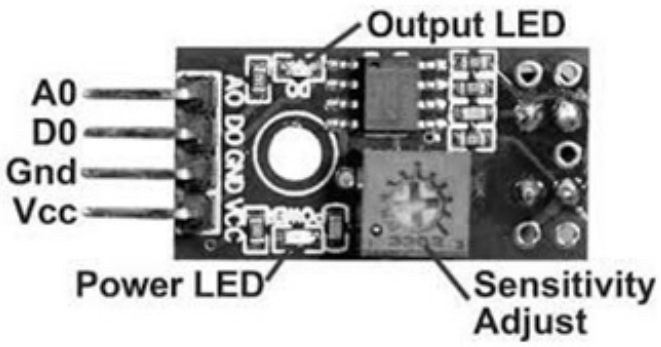



Figure 2.3: A TCRT5000 IR Sensor Module mainly includes parts like TCRT5000 IR sensor, power LED, reflective sensor, adjustment of sensitivity, and output LED

between the setpoint and this value.

Proportional control (P) applies a correction proportional to the current error. Simply proportional adjustment is the quickest and easiest way, which is just a proportional coefficient ($Kp$) times error, described as equation (2.2).

$$P = Kp \cdot error \tag{2.2}$$

Integral control (I) is similar; it accounts for accumulated past errors. Its difference is that each time the integral coefficient ($Ki$) times the total sum of all errors calculated in the history, described as equation (2.3).

$$I = Ki \cdot \int_0^t error \cdot dt \tag{2.3}$$

And derivative control (D), as its name suggests, is a derivative coefficient ($Kd$) times the deviation between this error and the last error. It responds to the error rate of change, described as equation (2.4) where dt refers to the time offset.

$$D = Kd \cdot \frac{error}{dt} \tag{2.4}$$

The total PID control is all three adjustments added together. All equations above are described mathematically, but are challenging for computers to understand. In coding environment, they need to be changed *discretely*, which, in total, can be described as equation (2.5), where $e(k)$= position error related to time $t$. In car situations, PD control is better than full PID control because integral term accumulates historical error causes overshoot.

$$u(k) = K_p e(k) + K_i \sum_{\tau=0}^{k} e(\tau) + K_d(e(k) - e(k-1)) \tag{2.5}$$

With only PID control, Lobster is still easy to veere off-track. We also applied a not-found-line function to overcome this issue. The function keeps recalling the last direction of deviation, and once Lobster loses its direction, it keeps rotating toward the memorized direction until any sensor detects a black line.

On Lobster, three TCRT5000 IR sensors are mounted in parallel with a 1.2 cm inter-sensor spacing. The voltage response of a single sensor exhibits a piecewise-linear characteristic, combining two distinct linear segments. Therefore, we construct a voltage-position graph figure 2.4 to analyze the sensor behavior.

The Left sensor's voltage minus the right yields a short linear segment around the origin $o$. The middle sensor's data verifies the peak values detected by the left/right sensors. Once the middle sensor cannot see black lines, we mirror the outermost sensor data to artificially extend the linear region.

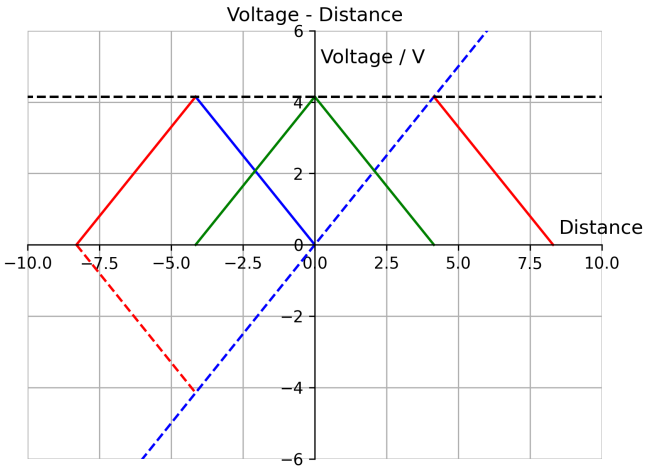



Figure 2.4: Voltage-position relationship of three TCRT5000 IR sensors. Dashed lines show the mirrored extension for linearity enhancement.

### *2.2.3 Results*

Lobster successfully completed the line-following test with 100% accuracy, as demonstrated in figure 2.5. The robot maintained a tracking speed of 0.3 m/s on curved paths and 0.5 m/s on straight segments.

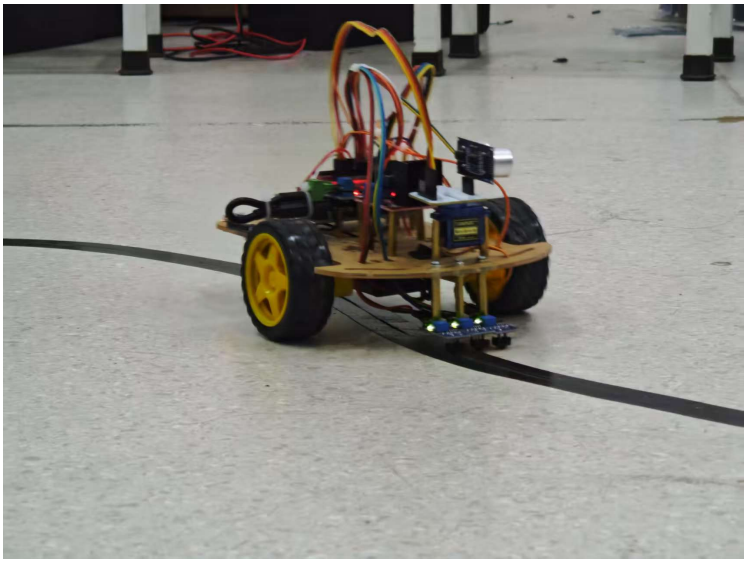


Figure 2.5: Lobster autonomously tracking a 3-cm-wide black line during the test.

## 2.3 Obstacle Avoidance

### *2.3.1 Theory and Technical Background*

In this experiment, *HC-SR04* and *SG90* are the two main sensors, showed in figure 2.6.

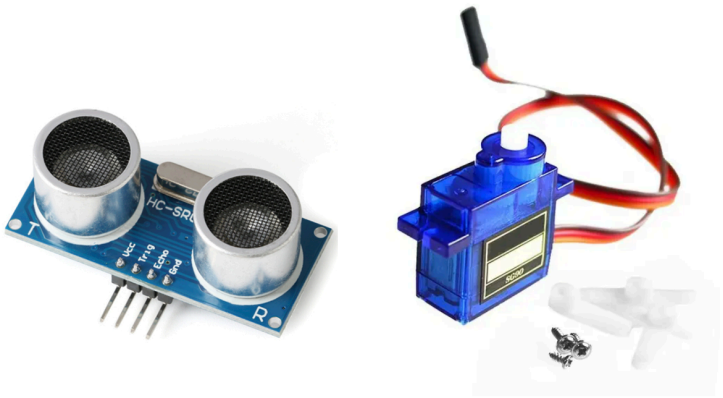


Figure 2.6: HC-SR04 ultrasonic sensor and SG90 servo motor diagram; pins of HC-SR04 are arranged from left to right as: *VCC*, *Trig*, *Echo* and *GND*.

The HC-SR04 ultrasonic sensor mainly consists of a transmitter and a receiver. This sensor has a measurement range of 2 cm to 400 cm with an accuracy of 0.3 cm. [2] Two pins are particularly important:

- The *Trig pin* serves as a trigger. Only a single high-level pulse needs to be applied to this pin during each measurement cycle.

- The *Echo pin* outputs a high-level signal when the HC-SR04 emits ultrasonic waves. This signal remains high until the sensor receives the reflected echo.

The HC-SR04 transmitter emits ultrasonic waves in one direction at a time. When these waves encounter an obstacle, they are reflected back and detected by the receiver. The distance between the HC-SR04 and the obstacle can be calculated by measuring the time difference between the emission and reception of the ultrasonic wave, combined with the speed of sound, as described in equation (2.6):

$$d = \frac{v \cdot \Delta t}{2} \tag{2.6}$$

where $d$ is the distance, $v$ is the speed of the ultrasonic wave, and $\Delta t$ is the time difference. During this process, the Echo pin's signal changes as follows:

- High level when the ultrasonic wave is emitted;
- Low level when the reflected wave is received.

The SG90 servo is a angle servo driver, suitable for control systems that require continuous angle changes and the ability to maintain a specific angle.

### *2.3.2 Method*

We designed a `carDrive` function for obstacle avoidance using a finite state machine. It processes sensor inputs and executes sequenced maneuvers when obstacles are detected. The complete logic of this function is shown in figure 2.7.

**Normal Operation**

In practice, Lobster may encounter obstacles from multiple directions, primarily from the left, front, and right. We mounted an SG90 servo motor on Lobster's front, with an HC-SR04 ultrasonic sensor attached. This allows the sensor to rotate and scan a wider area.

The SG90 servo rotates in 90-degree increments, performing a scanning motion at each position. Three distance measurements are obtained by: Front distance directly triggers avoidance maneuvers; Left and right distances are processed using equation (2.7):

$$\text{sideSafeDist} = \frac{\text{sideDist} \cdot \text{frontDist}}{\sqrt{\text{sideDist}^2 + \text{frontDist}^2}} \tag{2.7}$$

where: *sideSafeDist* is the effective clearance distance, *sideDist* represents left or right sensor readings, and *frontDist* is the front sensor reading

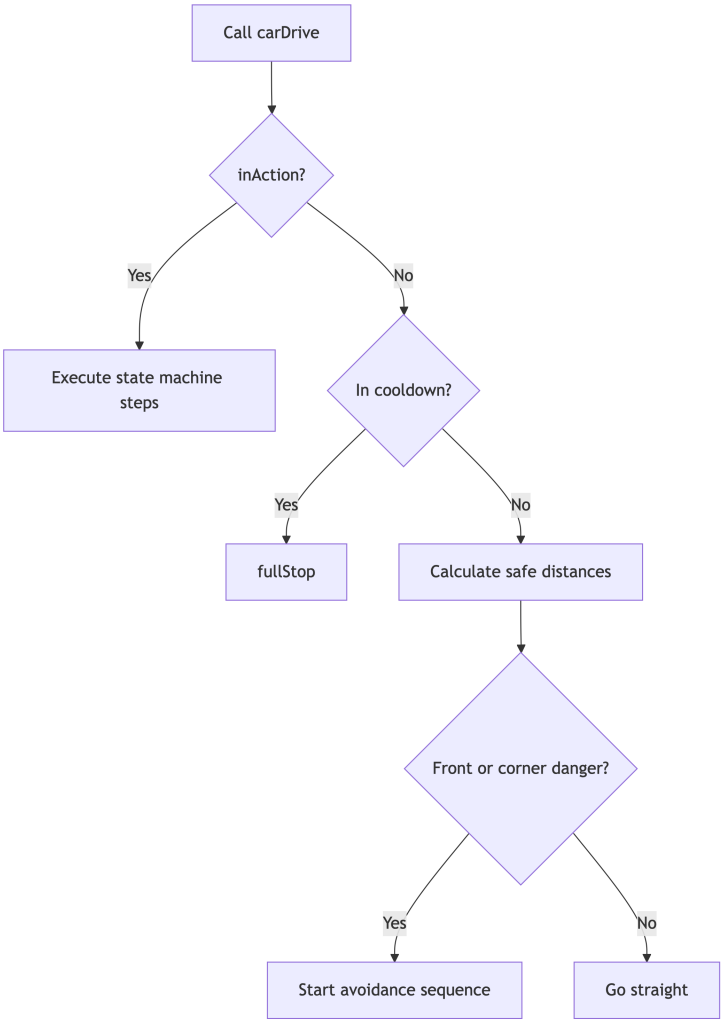



Figure 2.7: Flowchart depicting the logical structure of the `carDrive` function.

The vehicle moves straight when all clearance distances are safe. If any calculated distance falls below the threshold, the system transitions to avoidance mode.

**Avoidance Maneuvers**

**`STEP_BACK`**: First, reverse to create maneuvering space. Then check if sufficient clearance exists for a right turn (our designated avoidance direction). If adequate space exists, transition to `STEP_TURN_RIGHT_90`. Otherwise, proceed to `STEP_BACK_EXTRA`.

**`STEP_BACK_EXTRA`**: Perform additional reverse movement. Then transition to `STEP_TURN_RIGHT_90`.

**`STEP_TURN_RIGHT_90`**: Execute 90-degree right turn. Then proceed to `STEP_DONE`.

**`STEP_DONE`**: Complete full stop and restart the control loop.

### *2.3.3 Results*

The safety thresholds were empirically determined through iterative testing. The side clearance distance was set to 25 cm to account for gradual distance reduction during lateral approach, while the frontal clearance used a more conservative 40 cm threshold to ensure adequate reaction time for head-on obstacles. For turning maneuvers, a minimum 40 cm clearance was required to execute 90ř right turns safely.

Motion timing parameters were carefully calibrated, with each reverse segment lasting 100 ms and right turns completing in 300 ms, optimized for the gearbox motor's response characteristics. In final testing within a 3 m $\times$ 3 m arena, the Lobster demonstrated consistent performance.

## 2.4 Environment Sensing

### *2.4.1 Theory and Technical Background*

Give a brief description of the background theory for the experiment being tackled. Please cite appropriate work. Feel free to add figures into this section, where each figure and each equation may reduce the number of words you use.

### *2.4.2 Method*

Describe the technical procedures to collect the results. Add figures into this section, where each figure and each equation may reduce the number of words you use.

### *2.4.3 Results*

Add all appropriate figures and analysis. No formal word count for this section, however, please keep text as concise as possible.

# Chapter 3

# Conclusions

Please give the conclusions from the characterisations including a summary of the results collected.

# Appendix A

## Contribution Statement

Appendices, labelled A, B etc., should be treated as additional chapters and should normally follow the main text. Appendices may consist of supporting material of considerable length or of lists, documents, commentaries, tables or other evidence that if included in the main text, would interrupt its flow. The style of appendices must be consistent with the style of the main text. Long appendices may be divided into sections, labelled as Appendix A.1 etc., with corresponding subsection numbering, which must be entered in the table of contents. Alternatively, short appendices may be attached to individual chapters, as an extra section with a heading of style 3.7 Appendix

The contributions of each team member are as follows:

– Yuwei Zhao: Responsible for data processing and experiment implementation.

– Yilin Zhang: Contributed to the initial draft and collected supporting images.

– Haohao Qi: Wrote the final report and participated in experiment implementation.

# Appendix B

# Source Code

## B.1 Lab 3: Light Seeking

See GitHub Repository.

### *B.1.1 lightToward.cpp*

```
/* 4PIN-MODE */
int in1Pin = 5;
int in2Pin = 6;
int in3Pin = 9;
int in4Pin = 10;

// tail lights
int leftLight = 2;
int rightLight = 8;

// speed setting
int base_Speed = 120;
int decelerated_Speed = 110;
int accelerated_Speed = 170;

// fluctuating rate
float leftComp = 1.00;
float rightComp = 1.00;

// ldr offset
int left_offset = 10;
int right_offset = -20;
int expect_Gap = left_offset - right_offset;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  pinMode(leftLight, OUTPUT);
  pinMode(rightLight, OUTPUT);

  // Initialize
  analogWrite(in1Pin, 0);
  analogWrite(in4Pin, 0);
  digitalWrite(in2Pin, LOW);
  digitalWrite(in3Pin, LOW);
  digitalWrite(leftLight, LOW);
  digitalWrite(rightLight, LOW);
}

void loop() {
  // put your main code here, to run repeatedly:
  boolean reverse = 0;
```

```
47
48    int sensorValue_LEFT = analogRead(A5);
49    int sensorValue_RIGHT = analogRead(A0);
50    int GAP = abs((sensorValue_LEFT + left_offset) - (sensorValue_RIGHT +
      right_offset));
51
52    Serial.print("LEFT: ");
53    Serial.print(sensorValue_LEFT);
54    Serial.print("\t RIGHT: ");
55    Serial.print(sensorValue_RIGHT);
56    Serial.print("\t GAP: ");
57    Serial.println(GAP);
58
59    Turn(reverse, sensorValue_LEFT, sensorValue_RIGHT, GAP);
60
61    delay(10); // read arguments every 10ms
62  }
63
64  // Left Wheel CONTROL
65  void setLeft(boolean reverse, int speed) {
66    speed = constrain(speed, 0, 255);
67    digitalWrite(in2Pin, reverse);
68    analogWrite(in1Pin, speed);
69  }
70
71  // Right Wheel CONTROL
72  void setRight(boolean reverse, int speed) {
73    speed = constrain(speed, 0, 255);
74    digitalWrite(in3Pin, !reverse);
75    analogWrite(in4Pin, 255 - speed);
76  }
77
78  // car Turn CONTROL - LIGHT Toward
79  void Turn(boolean reverse, int Left, int Right, int Gap) {
80     // If light exists
81    if(Left < 1019 || Right < 1019) {
82      if(Gap >= expect_Gap) {
83        if(Left < Right) { // Turn LEFT
84          setLeft(reverse, decelerated_Speed * leftComp);
85          setRight(reverse, accelerated_Speed * rightComp);
86          digitalWrite(rightLight, LOW);
87          digitalWrite(leftLight, HIGH);
88        }
89        if(Right < Left) { // Turn RIGHT
90          setLeft(reverse, accelerated_Speed * leftComp);
91          setRight(reverse, decelerated_Speed * rightComp);
92          digitalWrite(leftLight, LOW);
93          digitalWrite(rightLight, HIGH);
94        }
95      } else { // Go FORWARD
96        setLeft(reverse, base_Speed * leftComp);
97        setRight(reverse, base_Speed * rightComp);
98        digitalWrite(leftLight , HIGH);
99        digitalWrite(rightLight, HIGH);
100     }
101   } else { // STOP
102     setLeft(reverse, 0);
103     setRight(reverse, 0);
104     digitalWrite(leftLight, LOW);
105     digitalWrite(rightLight, LOW);
106   }
107 }
```

## B.2 Lab 4: Line Following

### *B.2.1 dataTrans.h*

```
#ifndef DataTransfer_H
#define DataTransfer_H

#include <ADS1115_WE.h>
#include <Wire.h>

extern ADS1115_WE adc;

struct IRStatus {
  float left;
  float middle;
  float right;
};

float readChannel(ADS1115_MUX channel) {
  adc.setCompareChannels(channel);
  return adc.getResult_V();
}

IRStatus statusDetect() {
  IRStatus status;
  status.left = readChannel(ADS1115_COMP_0_GND);
  status.middle = readChannel(ADS1115_COMP_1_GND);
  status.right = readChannel(ADS1115_COMP_2_GND);

  Serial.print("Left: ");
  Serial.print(status.left);
  Serial.print(" V; Middle: ");
  Serial.print(status.middle);
  Serial.print(" V; Right: ");
  Serial.print(status.right);
  Serial.println(" V");

  return status;
}

#endif
```

### *B.2.2 Motion_setting.h*

```
#ifndef MotionSetting_h
#define MotionSetting_h

#define leftComp 0.99
#define rightComp 1.00

/* 4pin - mode */
int in1Pin = 4;
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
```

```
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int leftSpeed, int rightSpeed) {
  setLeft(reverse, leftSpeed * leftComp);
  setRight(reverse, rightSpeed * rightComp);
}

// Turn LEFT
void turnLeft(bool leftReverse, bool rightReverse, int leftSpeed, int
rightSpeed) {
  setLeft(leftReverse, leftSpeed * leftComp);
  setRight(rightReverse, rightSpeed * rightComp);
}

// Turn RIGHT
void turnRight(bool leftReverse, bool rightReverse, int leftSpeed, int
rightSpeed) {
  setLeft(leftReverse, leftSpeed * leftComp);
  setRight(rightReverse, rightSpeed * rightComp);
}

#endif
```

### *B.2.3 Motion_control.h*

```
#ifndef MotionControl_H
#define MotionControl_H

#include "dataTrans.h"

#define THRESHOLD 1.0

float cal_error(IRStatus);
float calculate_pid(float);
void motor_control(float);
void lostLineHandling(float);

const int baseSpeed = 105;
const float max_pid = 60.0;

float Kp = 5.6;
float Ki = 0;
float Kd = 5;
```

```
19
20 int lostCount = 0;
21
22 float last_valid_error = 0;
23
24 void carDrive() {
25   IRStatus status = statusDetect();
26
27   float error = cal_error(status);
28
29   if(error == 666) {
30     lostCount++;
31     if(lostCount >= 10) { // 10
32       lostLineHandling(last_valid_error);
33       lostCount = 0;
34     }
35   } else {
36     lostCount = 0;
37     float pid_val = calculate_pid(error);
38     motor_control(pid_val);
39   }
40
41   Serial.print("Error: ");
42   Serial.println(error);
43 }
44
45 void lostLineHandling(float last_error) {
46   unsigned long startTime = millis();
47
48   if (last_error < 0) {
49     turnLeft(false, true, 55, 115);
50   } else {
51     turnRight(true, false, 115, 55);
52   }
53
54   while (millis() - startTime < 100) {
55     IRStatus current = statusDetect();
56
57     if (current.middle > THRESHOLD || current.left > THRESHOLD ||
     current.right > THRESHOLD) {
58       fullStop();
59       return;
60     }
61   }
62
63   fullStop();
64 }
65
66 float cal_error(IRStatus status) {
67   float error = status.left - status.right;
68   if (status.middle <= THRESHOLD) {
69     if (status.left > THRESHOLD) {
70       error = 9 - status.left;
71     } else if (status.right > THRESHOLD) {
72       error = status.right - 9;
73     } else {
74       error = 666;
75     }
76   }
77   return error;
78 }
79
80 float calculate_pid(float error) {
81   static float I = 0;
82   static float previous_error = 0;
83
84   if (error != 666) {
```

```
    last_valid_error = error;
  }

  float P = error;
  I += error;
  I = constrain(I, -25, 25);
  float D = error - previous_error;

  float PID_value = (Kp * P) + (Ki * I) + (Kd * D);

  previous_error = error;

  return constrain(PID_value, -max_pid, max_pid);
}

void motor_control(float PID_value) {
  int left_motor_speed = baseSpeed - PID_value;
  int right_motor_speed = baseSpeed + PID_value;

  left_motor_speed = constrain(left_motor_speed, 50, 200);
  right_motor_speed = constrain(right_motor_speed, 50, 200);

  goStraight(false, left_motor_speed, right_motor_speed);
}

#endif
```

### *B.2.4 main.cpp*

```
#include "motion_setting.h"
#include "motion_control.h"
#include "dataTrans.h"

#define I2C_ADDRESS 0x48

ADS1115_WE adc = ADS1115_WE(I2C_ADDRESS);

void setup() {
  Wire.begin();
  Serial.begin(9600);

  if(!adc.init()) {
    Serial.println("ERROR: ADS1115 connected failed!");
  }

  adc.setConvRate(ADS1115_860_SPS);
  adc.setVoltageRange_mV(ADS1115_RANGE_6144);
  adc.setMeasureMode(ADS1115_CONTINUOUS);

  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  // Light Driver
  pinMode(leftLightPin, OUTPUT);
  pinMode(rightLightPin, OUTPUT);
}

void loop() {
  carDrive();
}
```

## B.3 Lab 7: Obstacles Avoiding

### *B.3.1 Motion_setting.h*

```
#ifndef MotionSetting_h
#define MotionSetting_h

/* 4pin - mode */
int in1Pin = 4;
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

float leftComp = 1.00;
float rightComp = 1.00;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int speed) {
  setLeft(reverse, speed);
  setRight(reverse, speed);
}

// Go BACK
void goBack(bool reverse, int speed) {
  setLeft(!reverse, 255 - speed * leftComp);
  setRight(!reverse, 255 - speed * rightComp);
}

// Turn LEFT
void turnLeft(bool reverse, int speed) {
  setLeft(!reverse, 255 - speed * leftComp);
  setRight(reverse, speed * rightComp);
}

// Turn RIGHT
```

```
void turnRight(bool reverse, int speed) {
  setLeft(reverse, speed * leftComp);
  setRight(!reverse, 255 - speed * rightComp);
}

#endif
```

### *B.3.2 Motion_control.h*

```
#ifndef MotionControl_h
#define MotionControl_h

#define front_border_VALUE 40
#define side_border_VALUE 40
#define corner_safe_VALUE 25

int baseSpeed = 200;
int turnSpeed = 100;
int deceleratedSpeed = 110;

bool reverse = false;

enum ActionStep {
  STEP_IDLE,
  STEP_BACK,
  STEP_BACK_EXTRA,
  STEP_TURN_RIGHT_90,
  STEP_DONE
};

ActionStep currentStep = STEP_IDLE;
unsigned long actionStartTime = 0;
unsigned long lastActionEndTime = 0;
bool inAction = false;
bool recentlyAvoidedCorner = false;

void carDrive(float leftDist, float frontDist, float rightDist) {
  if (!inAction) {
    if(recentlyAvoidedCorner && millis() - lastActionEndTime < 800) {
      fullStop();
      return;
    } else {
      recentlyAvoidedCorner = false;
    }
  }

  if (inAction) {
    unsigned long now = millis();
    switch (currentStep) {
      case STEP_BACK:
        if (now - actionStartTime < 100) {
          goBack(reverse, deceleratedSpeed);
        } else {
          if (rightDist <= side_border_VALUE) {
            currentStep = STEP_BACK_EXTRA;
          } else {
            currentStep = STEP_TURN_RIGHT_90;
          }
          actionStartTime = now;
        }
        return;

      case STEP_BACK_EXTRA:
        if (now - actionStartTime < 100) {
          goBack(reverse, deceleratedSpeed);
```

```
        } else {
          currentStep = STEP_TURN_RIGHT_90;
          actionStartTime = now;
        }
        return;

      case STEP_TURN_RIGHT_90:
        if (now - actionStartTime < 300) {
          turnRight(reverse, turnSpeed);
        } else {
          currentStep = STEP_DONE;
          actionStartTime = now;
        }
        return;

      case STEP_DONE:
        fullStop();
        inAction = false;
        currentStep = STEP_IDLE;
        lastActionEndTime = millis();
        recentlyAvoidedCorner = true;
        return;

      default:
        fullStop();
        inAction = false;
        currentStep = STEP_IDLE;
        return;
    }
  }

  float leftSafeDist = (leftDist * frontDist) / sqrt(pow(leftDist, 2) +
  pow(frontDist, 2));
  float rightSafeDist = (rightDist * frontDist) / sqrt(pow(rightDist, 2) +
  pow(frontDist, 2));

  fullStop();

  if (leftSafeDist <= corner_safe_VALUE || rightSafeDist <=
  corner_safe_VALUE) {
    currentStep = STEP_BACK;
    inAction = true;
    actionStartTime = millis();
    return;
  }

  if (frontDist <= front_border_VALUE) {
    currentStep = STEP_BACK;
    inAction = true;
    actionStartTime = millis();
    return;
  }

  goStraight(reverse, baseSpeed);
}

#endif
```

### *B.3.3 Servo_control.h*

```
#ifndef EngineControl_h
#define EngineControl_h

#include <Servo.h>

```

```
#define SERVO_INTERVAL 150

Servo MyServo;

int angles[] = {0, 90, 180, 90};
int curr = 0;
int lastAngle = angles[0];

unsigned long lastServoTime = 0;

int servoRotate() {
  unsigned long currentServoTime = millis();

  if(currentServoTime - lastServoTime >= SERVO_INTERVAL) {
    curr = (curr + 1) % 4;
    int angle = angles[curr];
    MyServo.write(angle);

    lastServoTime = currentServoTime;
    lastAngle = angle;
  }

  return lastAngle;
}

#endif
```

### *B.3.4 Ultrasound_control.h*

```
#ifndef Ultrasound_h
#define Ultrasound_h

int trigPin = 3;
int echoPin = 11;

float duration;

float distanceDetect() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  float distance = (duration * 0.0343) / 2;

  return distance;
}

#endif
```

### *B.3.5 main.cpp*

```
#include "motion_setting.h"
#include "motion_control.h"
#include "servo_control.h"
#include "ultrasound_control.h"

#define ANGLE_INTERVAL 30

float leftDist = 0;
float frontDist = 0;
```

```
float rightDist = 0;

unsigned long lastAngleTime = 0;

bool isWaiting = false;

void setup() {
  Serial.begin(9600);
  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
  // Light Driver
  pinMode(leftLightPin, OUTPUT);
  pinMode(rightLightPin, OUTPUT);
  // Engine Driver
  MyServo.attach(10);
  // Ultrasound Driver
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  int angle = servoRotate();
  int distance = distanceDetect();

  unsigned long currentAngleTime = millis();

  if(!isWaiting) {
    switch(angle) {
      case 0:
        leftDist = distance;
        Serial.print("Left: ");
        Serial.println(leftDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
      case 90:
        frontDist = distanceDetect();
        Serial.print("Front: ");
        Serial.println(frontDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
      case 180:
        rightDist = distance;
        Serial.print("Right: ");
        Serial.println(rightDist);
        isWaiting = true;
        lastAngleTime = currentAngleTime;
        break;
    }
    carDrive(leftDist, frontDist, rightDist);
  } else if(currentAngleTime - lastAngleTime >= ANGLE_INTERVAL) {
    isWaiting = false;
  }
}
```

## B.4 Lab 8: Environment Measurement

### *B.4.1 MotionDetect.h*

```
#ifndef MotionDetect_H
#define MotionDetect_H

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <math.h>

extern Adafruit_MPU6050 mpu;

struct variables {
  float x;
  float y;
  float z;
};

extern variables accData;
extern variables gyroData;

void getVariables() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  accData = { a.acceleration.x - 0.40, a.acceleration.y + 0.16,
  a.acceleration.z + 0.560 };
  gyroData = { g.gyro.x + 0.02, g.gyro.y - 0.06, g.gyro.z + 0.02 };
}

void mpuPrint() {
  getVariables();

  Serial.println("------------- MPU6050  -------------");

  Serial.println("| ----- Accelerations (m/s^2) ---- |");
  Serial.print("|    x: "); Serial.print(accData.x);
  Serial.print(", y: "); Serial.print(accData.y);
  Serial.print(", z: "); Serial.print(accData.z);
  Serial.println("    |");

  Serial.println("| --------- Gyro (rad/s) --------- |");
  Serial.print("|    x: "); Serial.print(gyroData.x);
  Serial.print(", y: "); Serial.print(gyroData.y);
  Serial.print(", z: "); Serial.print(gyroData.z);
  Serial.println("   |");

  Serial.println("-------------------------------------");
}

#endif
```

### *B.4.2 MotionSetting.h*

```
#ifndef MotionSetting_h
#define MotionSetting_h

#define leftComp 0.99
#define rightComp 1.00

/* 4pin - mode */
int in1Pin = 4;
```

```
int in2Pin = 5;
int in3Pin = 7;
int in4Pin = 6;

// tail lights
int leftLightPin = 2;
int rightLightPin = 8;

// Left Wheel CONTROl
void setLeft(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in1Pin, reverse);
  analogWrite(in2Pin, speed);
  digitalWrite(leftLightPin, reverse ? LOW: HIGH);
}

// Right Wheel CONTROL
void setRight(bool reverse, int speed) {
  speed = constrain(speed, 0, 255);
  digitalWrite(in3Pin, reverse);
  analogWrite(in4Pin, speed);
  digitalWrite(rightLightPin, reverse ? LOW: HIGH);
}

// STOP
void fullStop() {
  digitalWrite(in1Pin, LOW);
  analogWrite(in2Pin, 0);
  digitalWrite(in3Pin, LOW);
  analogWrite(in4Pin, 0);
  digitalWrite(leftLightPin, LOW);
  digitalWrite(rightLightPin, LOW);
}

// Go STRAIGHT
void goStraight(bool reverse, int speed) {
  setLeft(reverse, speed * leftComp);
  setRight(reverse, speed * rightComp);
}

#endif
```

### *B.4.3 MotionControl.h*

```
#ifndef MotionControl_H
#define MotionControl_H

#include "MotionSetting.h"

bool reverse = false;
int baseSpeed = 150;

bool isLifted = false;

const float LIFT_THRESHOLD = 8.5f;
const float GROUND_THRESHOLD = 10.0f;
const unsigned long DEBOUNCE_TIME = 250;

void carDrive() {
  getVariables();
  static unsigned long lastStateChangeTime = 0;
  static bool stableState = false;

  float acc = sqrt(pow(accData.x, 2) + pow(accData.y, 2) + pow(accData.z,
  2));
```

```

  bool stateChanged = false;
  unsigned long currentTime = millis();

  if (stableState) {
    if (acc >= GROUND_THRESHOLD) {
      if (lastStateChangeTime == 0) {
        lastStateChangeTime = currentTime;
      } else if (currentTime - lastStateChangeTime >= DEBOUNCE_TIME) {
        stableState = false;
        isLifted = false;
        stateChanged = true;
        lastStateChangeTime = 0;
      }
    } else {
      lastStateChangeTime = 0;
    }
  } else {
    if (acc <= LIFT_THRESHOLD) {
      if (lastStateChangeTime == 0) {
        lastStateChangeTime = currentTime;
      }
      else if (currentTime - lastStateChangeTime >= DEBOUNCE_TIME) {
        stableState = true;
        isLifted = true;
        stateChanged = true;
        lastStateChangeTime = 0;
      }
    } else {
      lastStateChangeTime = 0;
    }
  }

  if(isLifted) {
    goStraight(reverse, baseSpeed);
  } else {
    fullStop();
  }
}

#endif
```

### *B.4.4 FactorDetect.h*

```
#ifndef FactorDetect_H
#define FactorDetect_H

#include <Adafruit_BME280.h>
#include <Adafruit_Sensor.h>

extern Adafruit_BME280 bme;

float getTemperature() {
  return bme.readTemperature();
}

float getHumidity() {
  return bme.readHumidity();
}

float getPressure() {
  return bme.readPressure() / 100.0F;
}

void getBMEData(float &temp, float &hum, float &pres) {
```

```
  temp = getTemperature();
  hum = getHumidity();
  pres = getPressure();
}

void bmePrint() {
  float temp, hum, pres;
  getBMEData(temp, hum, pres);

  Serial.println("----------------------------- BME280
  ------------------------------");

  Serial.print("| Temprature: "); Serial.print(temp); Serial.print("");
  Serial.print(", Humidity: "); Serial.print(hum); Serial.print("%");
  Serial.print(", Air pressure: "); Serial.print(pres); Serial.println("hPa
  |");


  Serial.println("-------------------------------------------------------------------------");
}

#endif
```

### *B.4.5 LCDDemonstrate.h*

```
#ifndef LCDDemonstrate_H
#define LCDDemonstrate_H

#include <LiquidCrystal_I2C.h>
#include "FactorDetect.h"
#include "MotionDetect.h"

extern LiquidCrystal_I2C lcd;

void basicDemonstrate() {
  lcd.setCursor(1, 0);
  lcd.print("Bionic LOBSTER");
  lcd.setCursor(4, 1);
  lcd.print("Group #15");
}

void bmeDemonstrate() {
  static unsigned long lastScrollTime = 0;
  static int scrollIndex = 0;

  float temp, hum, pres;
  getBMEData(temp, hum, pres);

  String fullText = "T:" + String(temp, 2) + "C "
                  + "H:" + String(hum, 2) + "% "
                  + "P:" + String(pres, 2) + "hPa ";

  const int windowWidth = 16;

  fullText += "    " + fullText;

  unsigned long currentTime = millis();
  if (currentTime - lastScrollTime > 500) {
    lcd.setCursor(0, 0);
    lcd.print(fullText.substring(scrollIndex, scrollIndex + windowWidth));
    scrollIndex++;
    if (scrollIndex > fullText.length() - windowWidth) {
      scrollIndex = 0;
    }
    lastScrollTime = currentTime;
```

```
  }
}

void mpuDemonstrate() {
  static unsigned long lastScrollTime = 0;
  static int scrollIndex = 0;

  getVariables();

  String fullText = String("acc: ") + "x:" + String(accData.x, 2) + " "
                  + "y:" + String(accData.y, 2) + " "
                  + "z:" + String(accData.z, 2) + " "
                  + String("gyro: ") + "x: " + String(gyroData.x, 2) + " "
                  + "y:" + String(gyroData.y, 2) + " "
                  + "z: " + String(gyroData.z, 2) + " ";

  const int windowWidth = 16;

  fullText += "    " + fullText;

  unsigned long currentTime = millis();
  if (currentTime - lastScrollTime > 500) {
    lcd.setCursor(0, 1);
    lcd.print(fullText.substring(scrollIndex, scrollIndex + windowWidth));
    scrollIndex++;
    if (scrollIndex > fullText.length() - windowWidth) {
      scrollIndex = 0;
    }
    lastScrollTime = currentTime;
  }
}

#endif
```

### *B.4.6 main.cpp*

```
#include "FactorDetect.h"
#include "LCDDemonstrate.h"
#include "MotionDetect.h"
#include "MotionSetting.h"
#include "MotionControl.h"

#include <Wire.h>

Adafruit_BME280 bme;
Adafruit_MPU6050 mpu;
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  Wire.begin();
  Serial.begin(115200);
  while(!Serial);

  delay(1000);

  bme.begin(0x76);
  mpu.begin(0x68, &Wire);
  lcd.init();
  lcd.backlight();

  // Wheel Driver
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
```

```
  // Light Driver
  pinMode(leftLightPin, OUTPUT);
  pinMode(rightLightPin, OUTPUT);
}

variables accData;
variables gyroData;

unsigned long lastPrintTime = 0;

void loop() {
  carDrive();

  static bool basicShown = false;
  if (!basicShown) {
    basicDemonstrate();
    basicShown = true;
    delay(2500);
    lcd.clear();
  }

  bmeDemonstrate();
  mpuDemonstrate();

  static unsigned long lastPrintTime = 0;
  unsigned long currentTime = millis();
  if(currentTime - lastPrintTime >= 1000) {
    bmePrint();
    mpuPrint();
    Serial.println();

    lastPrintTime = currentTime;
  }
}
```

## References


[1] *TCRT5000(L) Reflective Optical Sensor with Transistor Output*. Document Number: 83760, Revision A4. Vishay Telefunken. 2000.

[2] HandsOn Technology. *HC-SR04 Ultrasonic Sensor Module User Guide*. Version 1.0. 2021. URL: http://www.handsontec.com.